

Skip-Gram Word2Vec: Neural Network View and Equivalence to Probabilistic Formulation

1 Introduction

The Skip-Gram model in Word2Vec can be presented from two equivalent perspectives:

- A **probabilistic formulation**, commonly used in theoretical NLP literature.
- A **neural network formulation**, commonly used in applied machine learning and teaching.

Although the notation differs, both formulations describe **exactly the same model**. This document presents the neural network (NN) view and rigorously proves its equivalence to the probabilistic formulation.

2 Neural Network Architecture of Skip-Gram

Given:

- Vocabulary size: V
- Embedding dimension: N
- Center (target) word: w_c
- Context word: w_o

The Skip-Gram neural network consists of:

1. An **input layer** (one-hot encoded center word)
 2. A **hidden layer** (word embedding)
 3. An **output layer** (softmax over vocabulary)
-

2.1 Network Diagram

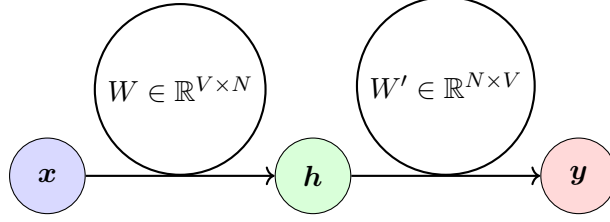


Figure 1: Neural network architecture of the Skip-Gram model

3 Forward Pass Computation

3.1 Input Layer

The input vector $\mathbf{x} \in \mathbb{R}^V$ is a one-hot encoding of the center word:

$$x_k = \begin{cases} 1 & \text{if } k = c \\ 0 & \text{otherwise} \end{cases}$$

3.2 Hidden Layer (Embedding Lookup)

The hidden layer output is computed as:

$$\mathbf{h} = W^\top \mathbf{x}$$

Since $\mathbf{x}$ is one-hot, $\mathbf{h}$ is simply the c -th row of W :

$$\mathbf{h} = \mathbf{v}_c$$

Thus, the matrix W stores the **input (center) word embeddings**.

3.3 Output Layer (Softmax)

For each word j in the vocabulary, the unnormalized score is:

$$z_j = \mathbf{u}_j^\top \mathbf{h}$$

where $\mathbf{u}_j$ is the j -th column of W' .

Applying softmax:

$$y_j = \frac{\exp(\mathbf{u}_j^\top \mathbf{h})}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{h})}$$

4 Equivalence to Probabilistic Skip-Gram

The probabilistic Skip-Gram model defines:

$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

Substituting:

$$\mathbf{h} = \mathbf{v}_c$$

we immediately obtain:

$$y_o = P(w_o \mid w_c)$$

Therefore, the neural network output is exactly the conditional probability defined by Skip-Gram.

5 Loss Function

The training objective is the negative log-likelihood:

$$\mathcal{L} = -\log P(w_o \mid w_c)$$

Using one-hot target vectors $\mathbf{t}$:

$$\mathcal{L} = -\sum_{j=1}^V t_j \log y_j$$

This is precisely the **categorical cross-entropy loss** used in neural networks.

6 Gradient Equivalence

6.1 Gradient w.r.t. Output Weights

For the output embedding $\mathbf{u}_j$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_j} = (y_j - t_j) \mathbf{h}$$

This expands to:

$$\sum_{j=1}^V P(j \mid c) \mathbf{v}_c - \mathbf{v}_c$$

which matches the probabilistic Skip-Gram gradient exactly.

6.2 Gradient w.r.t. Center Embedding

$$\frac{\partial \mathcal{L}}{\partial \mathbf{v}_c} = \sum_{j=1}^V (y_j - t_j) \mathbf{u}_j = \sum_{j=1}^V P(j \mid c) \mathbf{u}_j - \mathbf{u}_o$$

Again, this is identical to the gradient derived in the probabilistic formulation.

7 Conclusion

- The Skip-Gram neural network is a **log-linear model with softmax**.
- The input-hidden matrix W and hidden-output matrix W' correspond exactly to center and context embeddings.
- The NN loss is identical to the negative log-likelihood of Skip-Gram.
- All gradients and updates are mathematically equivalent.

Conclusion: *The neural network view and the probabilistic view of Skip-Gram are not two models — they are the same model written in different languages.*